FIDELITYLOOP: CLOSED-LOOP VALIDATION OF ELASTIC LLM-SERVING POLICIES

Anonymous Authors[1]

ABSTRACT

Replay models guide elastic LLM-serving policies for online requests and deadline-constrained offline jobs sharing GPUs, but accurate timing and aggregate resource estimates need not preserve deployment decisions. We present FidelityLoop, a closed-loop framework aligning replay and physical execution through shared policy interfaces, request identities, and lifecycle accounting. It supports rule-based policies and PPO training in replay, followed by frozen-policy validation on vLLM/SGLang; Vidur supplies an alternative replay service model. Measurements on two A100 GPUs and controlled replay contrasts reveal two mechanisms: startup timing can cross a controller's cooldown boundary and alter subsequent actions, while resource errors can cancel in aggregate despite temporal misalignment. Retrospective selection chooses a policy failing all six physical executions in one pool and a feasible policy costing 58.95% more than the cheapest feasible alternative in another. We then freeze predictions and choices before 36 executions with actual HTTP inference. Host-calibrated replay, a fixed 30 s proxy, and a timer diagnostic select the cheapest consistently feasible candidate in the eight-policy pool; reusing the archived startup estimate selects one costing 19.87% more, including provider GPU capacity. On full-day arrivals, locally calibrated replay still accepts a guarded policy that misses offline deadlines; model and four-replica studies test transfer. Request-aligned traces connect timing discrepancies to actions and deadlines. We release paired traces from 197 historical physical runs and companion records for the deployment extensions, and CPU analysis tools under explicit study-specific cost contracts.

1 INTRODUCTION

Elastic LLM serving can share GPU capacity between online requests and deadline-constrained offline jobs. Releasing instances saves resources, but delayed startup can jeopardize pending work. Queues, running work, and lifecycle transitions shape subsequent control decisions. Replay must capture when capacity becomes available, beyond request timing and total GPU seconds.

FidelityLoop links replay-based policy development and PPO training to physical validation through a shared policy contract (Figure 1). Retrospective selection picks a PPO policy failing all six runs in one tested pool and a feasible policy costing 58.95% more than the cheapest feasible alternative in another. In prospective HTTP execution, calibrated replay's frozen choice is cheapest among consistently feasible candidates in an eight-policy pool; an archived startup estimate selects one costing 19.87% more, including provider GPU capacity.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

The paper makes three contributions:

1. **A reusable closed-loop validation framework.** FidelityLoop supports rule-based policy development and PPO training in replay, followed by physical validation of frozen policies. Shared interfaces and request identities align service, control, and resource traces to check online/offline deadline classifications and candidate-policy choices.
2. **Mechanisms of replay-to-deployment disagreement.** Held-out physical runs distinguish service and lifecycle (aggregate and trajectory) fidelity. Event-level analysis and controlled replay contrasts expose startup–cooldown interactions and cancellation in aggregate resource error.
3. **Decision-level validation.** Retrospective selection admits failing policies or excludes cheaper feasible alternatives; prospective selection with generated HTTP responses tests whether a frozen choice succeeds physically. Physical studies cover rule-based and frozen PPO policies, full-day arrivals, two serving engines, a 14B model, quantized execution, and active four-replica control. A guard case study quantifies deadline protection and resource–latency trade-offs.

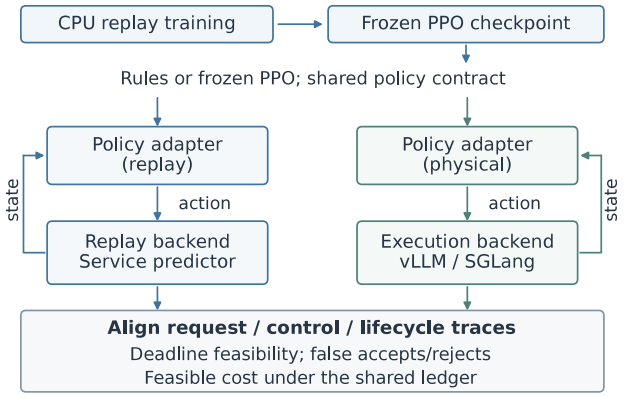


Figure 1. Rule-based policies and frozen replay-trained PPO checkpoints enter separate replay and physical control loops with shared input identities and a policy contract. Each adapter reads its own backend's causal state. Aligned traces expose errors in deadline predictions and policy choices, using physical outcomes as the reference.

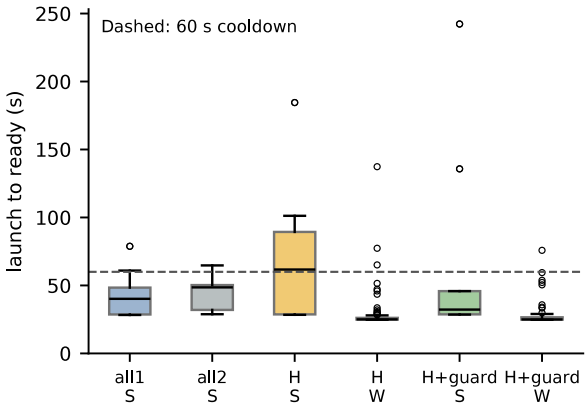


Figure 2. Observed startup times in the held-out runs. Labels use S for setup and W for window scale-out. The dashed line marks the 60-second controller cooldown; boxes summarize observed launch-to-ready times. Release and cleanup are separate ledger events. Collapsing these contexts into one endpoint can alter closed-loop actions.

We provide paired traces, CPU feasibility/cost tools, and fixed-input Vidur replay[1].

2 Background and Motivation

Mixed online and offline LLM workloads share a limited GPU pool, so elastic scheduling seeks resource savings subject to request deadlines. Physical evaluation of every candidate is costly.

Preliminary calibration reduced timing error for matched local requests but increased occupied-GPU-time error under dynamic scheduling, motivating the preregistered held-out study. Startup estimates become decision-sensitive near a controller's cooldown boundary. In a controlled replay contrast, 69.219 s and 30 s startup estimates straddle the 60 s cooldown and produce different capacity targets (Section 8.1). This motivates checking lifecycle timing through control actions and deadline outcomes. Figure 2 shows startup distributions from the held-out physical runs, separating setup from window scale-out.

3 Related Work

Simulation already informs deployment: Vidur and Charon guide configuration search, and InferLine couples profiling and simulation with SLO-aware provisioning (Agrawal et al., 2024a; Yang et al., 2026; Crankshaw et al., 2020). FidelityLoop adds a validation contract around policy rollouts: each backend drives decisions from its own causal state, while service and lifecycle models can change separately. Request-aligned accounting retains undispatched work and launch-to-release occupation, exposing false accepts/rejects and the cost of feasible choices. These decision checks complement configuration search.

Cold-start characterization decomposes initialization latency, while ServerlessLLM reduces startup overhead (Kabakibo et al., 2026; Fu et al., 2024). We study how timing errors interact with controller cooldown to change subsequent actions and deadlines; controlled replay varies startup with the backend and loading path fixed.

Clockwork targets predictable request timing; vLLM, SGLang, and Sarathi-Serve optimize execution (Gujarati et al., 2020; Kwon et al., 2023; Zheng et al., 2024; Agrawal et al., 2024b). Provisioning and scheduling connect workload estimates to SLOs (Zhang et al., 2019; 2023). ConServe optimizes online/offline co-serving within an inference engine (Qiao et al., 2024); FidelityLoop validates elastic decisions across replay and physical execution.

Policy design is complementary: Llumnix reschedules LLM requests (Sun et al., 2024); AAPA predicts serverless scaling needs (Zhang et al., 2025b); KIS-S trains PPO autoscaling policies in simulation for deployment (Zhang et al., 2025a). We use standard PPO (Schulman et al., 2017) to train and freeze policies, then trace incorrect replay predictions to their physical feasibility and cost consequences.

4 Problem and Metrics

The primary studies use two accelerators with online and offline deadlines of 60 s and 300 s. Online requests run locally or offload; offline jobs run locally. Earlier studies use a delay-only synthetic API sink; the prospective study

[1]FidelityLoop anonymous artifact.

uses actual HTTP inference. Controlled-window studies contain 120 offline jobs over 1,800 s of arrivals followed by 300 s of observation. The full-day study retains these deadlines over 24 h of arrivals and a 300 s drain; capacity studies use two and four independent replicas under the same deadline contract. The primary window cost under a fixed USD pricing scenario is

$$C = .001G + .002S + .001D + 10^{-6}I + 3\times10^{-6}B + .01M, \quad (1)$$

where G counts GPU seconds from launch to verified release within the window, S, D are starts and shutdowns, I, B are synthetic API input and output budgets, and M counts offline deadline misses, including unfinished requests whose deadlines have passed. Setup and cleanup are reported separately. The prospective study instead charges all local and provider GPU occupation in C_{sys} (Supplement Section R); the two ledgers are analyzed separately.

Completed means a recorded completion by the observation cutoff; it may be *on time* (finish time $\leq$ deadline) or *late* (finish time $>$ deadline). An unfinished request has no completed terminal by the cutoff. Unfinished or censored work is retained in the arrival denominator. The capacity gate G_F requires all2 (two GPUs throughout) to complete every online and offline request on time. P1 requires every offline request on time, at least 99% online requests on time, and no unfinished request; P1+ additionally requires every online request on time. In the original three-window study, P2 is a ratio-of-means saving against all1 (one GPU throughout) computed within each window and then averaged equally across the three windows.

Service fidelity measures request timing error; *lifecycle fidelity* measures aggregate and time-resolved resource and queue error; *feasibility fidelity* measures deadline outcomes, including false accepts/rejects. *Decision fidelity* checks physical feasibility of replay-selected policies and, for feasible selections, the cost gap to the lowest-cost feasible candidate in the tested pool. Agreement at one level can coexist with disagreement at another.

5 Workload construction

Online arrivals come from successful requests in BurstGPT (Wang et al., 2024). We read all 1,404,294 source rows and retain their identities and hashes. Without session identifiers, the split separates arrival times. Local requests execute fixed-corpus prompts with registered token budgets. Feasibility measures request deadlines; semantic answer correctness is separate.

The three held-out windows follow development windows with a 30-minute exclusion. A frozen 60 s arrival-count filter and deterministic hash ordering choose steady, recovery, and burst-offline windows; online thinning is fixed. Each receives 120 offline jobs at 15 s intervals with 256-input/256-output budgets. The supplement records selection rules, separation, and input hashes.

Table 1. Selected window arrival shape at 60 s bins. Each cell is raw/derived; CV and peak-to-mean use the full 1,800 s window, including empty bins.

Window	arrivals	CV	peak/mean	lag-1
steady	5461 / 1444	0.378 / 0.412	1.566 / 1.641	0.772 / 0.797
recovery	1141 / 334	1.444 / 1.452	3.550 / 3.862	0.906 / 0.892
burst-offline	663 / 177	1.951 / 1.935	6.652 / 6.780	0.810 / 0.754

Table 1 describes the selected windows at the 60 s scale, not the source population. The manifest's 1 s diagnostics do not define the classes.

The deployment study uses two separately frozen windows from the same pipeline, with no source-row overlap with the three held-out windows. Screening, engine, and 14B studies reuse them; studies are analyzed separately. Recovery contains 358 online and 120 offline arrivals (478 total); steady contains 944 online and 120 offline arrivals (1,064 total). Timestamps and offline construction remain fixed, with input identities recorded in the workload manifest.

A separate external-validity study uses Azure conversation arrival traces associated with Splitwise (Patel et al., 2024); its workload construction and load-boundary results are reported in the supplement.

The full-day study preserves thinned BurstGPT arrival times and source-derived, capped length budgets, with continuous controller state and frozen host-local Qwen 7B/vLLM calibration (supplement).

6 Experimental setup

We evaluate replay fidelity, replay-based policy selection reliability, and transfer across serving configurations and arrival patterns through common policy and accounting interfaces. The preregistered held-out study used two NVIDIA A100 80GB PCIe GPUs at a university computing facility, each running Qwen2.5-7B-Instruct FP16 with TP1 on vLLM 0.9.0 (Kwon et al., 2023), PyTorch 2.7.0+cu126 and Transformers 4.51.1. Four local slots per GPU, eager execution, chunked prefill, no prefix caching, and a dual-ready barrier define the execution contract. Per-run records and the supplement specify remaining versions, limits, readiness, and release checks.

The deployment-adapted model (Local) was calibrated and frozen before held-out execution. Legacy-refit is retained as a historical lifecycle reference; its contrast with Local is descriptive and does not isolate calibration.

The deployment and screening studies test replay judgments on frozen PPO checkpoints. Deployment executes five checkpoints on two locked windows (three repeats and two references: 32 runs). Screening adds six previously undeployed checkpoints (26 runs); a separate mixed-policy cohort has 38 runs. Checkpoint hashes identify shared policies; each cohort retains its own recorded lifecycle and hardware conditions.

Transfer studies test the same contract across engines, models, arrivals, and capacity. The engine study uses SGLang 0.4.8 on dual A100s (26 runs); the arrival study tests additional arrivals on that stack (27). Qwen2.5-14B/vLLM and 7B AWQ-Marlin add 12 runs each. Two/four-replica A800 studies add 36 runs with profiles frozen before execution. The supplement specifies the software, calibration, and repeat matrices.

The prospective study tests a choice frozen before physical execution. It uses two elastic local replicas and two fixed HTTP provider replicas on four same-host A800s, with the same 7B weights and decoding settings. Eight existing candidates (H, H+guard, six frozen PPO checkpoints) run on the two known deployment windows twice, plus four all2 references: 36 cells. Profiles, predictions, and selections are frozen before any formal setup; no checkpoint is retrained (Supplement Section R).

7 METHOD

Predictive replay and physical execution share a policy contract. A policy adapter maps causal queue and device observations to capacity targets; the guard can also choose dispatch priority. A service predictor supplies completion timing for admitted work. An execution backend implements launch, readiness, inference, draining, and verified release. Shared request identities align service, control, and resource traces; a CPU verifier reconstructs deadline feasibility and cost from these records. Each one-second tick releases due arrivals, reads backend-visible state, applies the policy with cooldown/guard, then changes capacity and dispatches. Completion, readiness, and release retain their backend timestamps; only ready, accepting devices admit work. Replay contrasts vary timing models while preserving these interfaces and accounting semantics. By default, ready local slots serve online requests before offline jobs. Online requests still waiting 10 s after arrival can offload. The earlier synthetic sink admits four concurrent requests per allocated GPU on a 5 s timer. In the prospective study, two fixed providers offer eight total slots; completion requires a validated generated HTTP response, with no timer padding. The guard can prioritize at-risk offline jobs; PPO changes capacity while retaining the default dispatch and API rules.

The policy and backend adapters have separate interfaces; the supplement reports their implementation footprint. CPU checks preserve historical request/event ledgers, and a CPU causal-replay check reproduces all 29,580 logged decisions from 16 same-runtime runs, validating adapter semantics rather than adding an independent physical cohort. A Vidur service backend (Agrawal et al., 2024a) supplies native scheduling and token execution while FidelityLoop retains admission, lifecycle transitions, and the frozen guard. A Qwen2.5-7B/A100 service screen precedes the retrospective policy comparison in Section 8.1; the supplement provides profiles and parameters.

Table 2. Frozen replay parameters in seconds, rounded to three decimals. Primary held-out models share a 69.363 s setup proxy outside the window ledger.

Model	Service	Startup	Shutdown
Share	equal-share	30.000	7.059
Lookup-69	measured lookup	69.219	7.059
Lookup-30	same lookup	30.000	7.059
Local	same lookup	33.497	2.224
Legacy-refit	same lookup	25.196	2.325

7.1 Replay models

The replay models share an executor but differ in service and lifecycle timing (Table 2). Share uses an equal-share service model and an assumed 30 s startup. Lookup-69 uses a measured service lookup and a historical 69.219 s startup proxy; Lookup-30 restores the 30 s assumption. Local combines the lookup with deployment-adapted lifecycle parameters; Legacy-refit retains a bounded refit from the earlier environment. Share–Lookup-30 isolates the service model, whereas Lookup-69–Lookup-30 isolates startup; Share–Lookup-69 changes both.

Share assumes 1024 input and 128 output tokens/s, shared equally among concurrent requests. The calibrated lookup uses median measured wall times per input/output/concurrency cell, interpolating in input length and concurrency (one to four requests per instance). These four lookup-based models share the values. Supplement Section B.1 details measurement sources, interpolation, and lifecycle fitting. Model bytes, tokenizer, workload identities, and the candidate grid are locked before held-out execution.

For the engine study, Lookup-SG replaces Lookup-69's startup proxy with the median of 20 SGLang scale-outs measured while the other GPU serves a fixed background load: 27.568 s. It retains Lookup-69's service lookup and shutdown parameter. Lookup-69, Lookup-30, and Lookup-SG predictions are frozen before the policy runs; the guard retains its separate safety estimates.

The arrival study reuses these models unchanged. The 14B

study freezes 14B-Hist and 14B-Local with freshly measured service, setup, shutdown, and guard parameters; only startup differs (69.219 versus 46.067 s).

For each model prediction and physical repeat of a window, the dimensionless lifecycle loss is

$$L = \frac{0.5|\Delta G_{\text{occupied}}| + 0.25|\Delta G_{\text{active}}|}{2T} + \frac{0.25|\Delta Q|}{TN}, \quad (2)$$

Here G_{occupied} sums per-GPU time from launch issuance to verified release, while G_{active} sums time in the active (ready-to-serve) state, including idle time; both are measured in GPU seconds. The difference Δ is prediction minus observation, $T = 2100$ s for formal windows, and N counts *all* online and offline arrivals. Q is the time integral of the released-but-undispatched queue, in request seconds. Unfinished requests remain in N and contribute queue time while queued up to cutoff. Two devices give the $2T$ normalization. We first average losses over repeats within each window, then weight windows equally; we do not pool requests or restrict N to matched-local support. L measures prediction error, separately from scenario cost C; the shorter development horizon is stated with that experiment.

7.2 PPO policy and training protocol

PPO tests whether replay error also changes learned control behavior, beyond H's hand-written thresholds. Training uses CPU replay with Lookup-69 or Lookup-30; physical validation follows checkpoint freezing. At each one-second decision boundary, its 30-dimensional causal observation contains normalized time and target state, queue counts and ages/slack, device lifecycle states and ages, running-job summaries, and the API queue. The capacity action proposes a target of 0, 1, or 2 GPUs; the registered 60 s cooldown filters that proposal before execution, while the offline action head is fixed to its sole legal value. The reward is the negative incremental scenario-ledger cost, so training trades occupied resources, lifecycle events, API use, and deadline misses under the declared contract. The policy observes this causal snapshot of replay state; future arrivals and physical outcomes are unavailable. We optimize the standard clipped PPO policy surrogate

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\big[\min\{r_t(\theta)\widehat{A}_t, \operatorname{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\}\big], \quad (3)$$

where $r_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\theta_{\text{old}}}(a_t \mid o_t)$. The total objective adds the clipped value loss and entropy bonus. The policy uses a shared two-layer 256-unit ReLU MLP with three capacity logits and a scalar value head; the offline head is masked to its sole legal action. Coefficients and optimization settings are recorded in the supplement.

Five frozen checkpoints cover both training environments: a reference and matched counterpart, two eligible rank-2 checkpoints, and a seed-103 best. Eligibility requires Lookup-30 P1+ in every validation window without guard; ordering uses mean cost, checkpoint index, then seed (supplement). PPO-1/2 were trained with Lookup-69; PPO-3–5 with Lookup-30, using three development regimes and paired seeds. The supplement records coordinates, inclusion rules, and a source-row isolation audit of training, validation, and deployment inputs.

7.3 Controller and guard

H and its guard separate deadline protection from learned-policy behavior. H uses up-threshold 8, down-threshold 0, and a 60 s cooldown. At each one-second tick, H reads queue and running counts plus its target/cooldown state, not the predictor's startup parameter. The guard's lifecycle-delay estimates use separately frozen safety parameters, distinct from the Lookup-69/Local simulator endpoints. The guard observes only released queues, device states, and running-job ages; future arrivals and true completion or readiness times are unavailable. Let P_t be the pending offline request IDs and b_t H's current target after its decision. The projection $F_j(k, r)$ heuristically estimates request j's finish with k devices and reservation setting r, using frozen lifecycle delays, running-job residuals, and concurrency-four service estimates. It schedules pending jobs by deadline then request ID over four slots per device. Online backlog precedes offline work unless reservation ($r = 1$) assigns one slot to offline jobs; without online backlog, offline work can use all slots. The supplement's "Guard implementation map" specifies device selection and overdue-estimate fallbacks.

For the full guard ($r = 1$), margin $m = 5$ s and previous risk-ID set A_{t-1}, the primary two-GPU update is

$$\begin{aligned} R_t &= \{j \in P_t : F_j(\max(1, b_t), 1) + m + 1 \geq d_j\}, \\ A_t &= (A_{t-1} \cap P_t) \cup R_t, \\ K_t &= \{k \in \{1, 2\} : \forall j \in P_t,\ F_j(k, 1) + m \leq d_j\}, \\ u_t &= \begin{cases} b_t, & A_t = \emptyset, \\ \max(b_t, \min K_t), & |A_t| > 0,\ |K_t| > 0, \\ 2, & |A_t| > 0,\ K_t = \emptyset. \end{cases} \end{aligned} \quad (4)$$

The four-replica configuration tests candidates $0, \ldots, 4$ and uses fallback four. Here d_j is the deadline. Triggering includes a one-second lookahead and uses $\geq$; candidate feasibility uses $\leq$ without that extra second. Empty K_t with active risk sets an `infeasible` flag. A latched ID is removed only when it leaves the pending queue, not when a later estimate becomes safe. The target never falls below *this tick's* H target. A guard raise can bypass H's cooldown and writes u_t and the current time back to H's target and last-change state.

While A_t is nonempty and both queues have work, dispatch reserves at most one offline slot. An already-running

offline job leaves other slots online-first. Otherwise, for head-request slacks $s_x = d_x - t - \widehat{p}_x$, online wins only if $s_{\text{on}} \le m$ and $s_{\text{on}} \le s_{\text{off}}$. Without online backlog, offline dispatch is unrestricted. No running request is preempted; later scale-out must wait for readiness.

The 26-configuration development search selected H+guard before held-out input access; its duplicate arm was merged in the formal matrix.

7.4 Evidence protocol

The held-out study evaluates all1, all2, H, and H+guard on three windows with three repeats per arm (36 runs). Original acceptance code and lock-bound sources recompute terminal counts, lifecycle occupation, and cost. Two operator-aborted attempts remain in history but are excluded from formal statistics; technically valid failures remain included. Registered gates, run/window metric units, and diagnostics are detailed in the supplement.

8 RESULTS

We first trace prediction errors to control actions, then test policy feasibility and selection, including a prospective HTTP deployment. Transfer studies probe the boundaries; the guard case explains deadline protection and its resource–latency trade-off.

8.1 Cross-level prediction fidelity

On fixed H, Local's occupied MAE is 67.94 GPU seconds versus 543.51 for Share (87.50% lower). Lookup-30's improvement over Lookup-69 implicates startup. Legacy-refit reduces equal-window L by 34.24% relative to Local using its earlier lifecycle snapshot.

In fixed all1/all2 tests, maximum service MAE is 0.064 s for all four lookup-based models, versus 3.016 s for Share. On dynamic arms, the minimum common support across model/route projections is 122 requests and the maximum common-support MAE is 0.129 s for Lookup-69, 0.112 s for Lookup-30, 0.110 s for Local, and 0.112 s for Legacy-refit; Share reaches 4.510 s. Common support is the intersection of request IDs completed locally in the compared predictions and physical runs; coverage and routing discrepancies are reported in the artifact.

Table 3 reports window-wise occupied-resource errors for fixed-H replay, distinct from the route-conditioned matched-ID checks.

The Lookup-69-versus-Share comparison reproduces the exploratory divergence: maximum matched-ID service MAE falls from 4.510 to 0.129 s (about 35-fold), while equal-window occupied-resource MAE rises from 543.51 to 891.98 GPU seconds (64.1%). With service fixed, Lookup-30 lowers occupied MAE to 76.19 GPU seconds, isolating startup. Model quality is therefore metric- and decision-dependent: the best matched-request timing need not give the best lifecycle or trajectory fidelity.

Table 3. Occupied-resource MAE (GPU seconds) by window and model. L is the equal-window lifecycle loss defined above.

Model	steady	recovery	burst	Occupied MAE	L
Share	1151.97	244.08	234.48	543.51	0.09403
Lookup-69	964.14	864.20	847.60	891.98	0.12162
Lookup-30	67.18	78.30	83.08	76.19	0.01353
Local	66.34	66.95	70.55	67.94	0.01353
Legacy-refit	43.51	65.74	69.34	59.53	0.00890

Table 4. Steady-window fidelity: means over the same three physical repeats. Resource errors are GPU seconds; target disagreement is a percentage of decision ticks.

Model	Total error	Trajectory error	Target diff. (%)
Lookup-30	67.18	114.32	1.03
Local	66.34	1390.50	66.73

Retrospective analysis of the frozen trajectories explains this endpoint effect and its limits (Supplement Section N). In all three windows, Lookup-69/Lookup-30 targets first diverge 60 s after a common instance start: Lookup-69 remains unready with higher queue/running pressure; Lookup-30 is ready. The 60 s contrast isolates this controlled simulator mechanism; six burst/recovery physical repeats first disagree in target with predictions at 3 s, before any window restart, showing that other state and routing differences can also precede a restart. Conversely, steady repeats 1 and 3 match Lookup-30 targets throughout despite state, routing, and occupation errors.

Aggregate agreement can mask temporal misalignment. In steady, Local and Lookup-30 have nearly equal total occupation MAE, but Local's mean time-integrated absolute error in occupied GPU count is 1390.504 versus 114.324 GPU seconds (Table 4). Positive and negative count errors cancel in the total; their timing still matters to control. Matching capacity targets does not ensure resource-trajectory agreement. The supplement retains all 36 pairs and the fixed comparison criterion.

Twelve retrospective CPU rollouts replace the service lookup with Vidur's profiled backend, retaining Local's lifecycle and the frozen guard. Against the same 36 physical runs, they preserve H's recovery deadline failures and the guard's success. In steady, 2.47% median occupied-resource APE coexists with only 33.0–35.5% target agreement: a

Table 5. Policy deployment on two A100 GPUs. Costs are mean scenario USD; R/S report P1+ passes/runs in recovery/steady. The last column is cost reduction relative to all2; † marks infeasible savings.

Arm	R	S	Cost R	Cost S	Cost ↓ (%)
all2	1/1	1/1	4.200	4.200	0
PPO-1	3/3	3/3	2.357	2.449	42.8
PPO-3	3/3	3/3	2.073	2.084	50.5
PPO-4	3/3	3/3	2.051	2.103	50.5
PPO-5	0/3	0/3	1.899	2.090	52.5†
PPO-2	0/3	0/3	2.730	2.950	32.4†

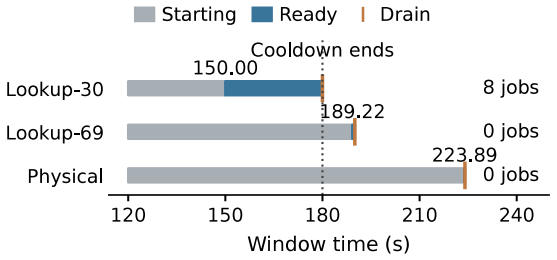


Figure 3. PPO-2's first recovery restart: readiness before cooldown expiry permits eight offline admissions in Lookup-30; Lookup-69 and physical repeat 1 drain without admissions. Labels mark ready times; cooldown starts at the target change near 120 s.

backend can recover feasibility while still misaligning actions (Supplement Section A.3).

8.2 Policy deployment and screening

The deployment study tests replay-selected PPO checkpoints in physical execution without retraining or reselection. In its re-provisioned runtime, 196 of 218 window restarts that reached readiness across the 30 PPO runs took over 60 s. All 32 cells are technically valid; the supplement retains four earlier technical failures. PPO-4 is the preregistered reference checkpoint.

Three PPO arms pass all six physical repeats: PPO-1, PPO-3, and PPO-4; both all2 anchors also pass. The three feasible PPO arms have scenario costs 42.8–50.5% below all2, but this comparison is conditional on the declared ledger and synthetic API sink. PPO-2 averages 4–7 on-time offline requests out of 120 and leaves about 80 unfinished, despite lower cost. PPO-5 completes the offline stream, but roughly 11 of 120 offline requests miss their deadline per repeat, so its lower cost is not a feasible saving. All online requests meet their deadlines in every cell, yet offline failures make PPO-2 and PPO-5 infeasible. Online responsiveness alone therefore cannot validate savings for mixed workloads.

Table 6 compares frozen predictions with physical outcomes.

Table 6. Agreement over physical cells and cost MAE (scenario USD). Mixed-policy and deployment include anchors and failures; screening includes 24 policy runs. Deployment reuses 12 distinct predictions per replay model across 32 cells.

Study	Replay	Agreement	Cost MAE
Mixed-policy	Lookup-69	36/38	0.0969
Mixed-policy	Lookup-30	21/38	0.6998
Deployment	Lookup-69	32/32	0.0804
Deployment	Lookup-30	20/32	0.4657
Screening	Lookup-69	18/24	0.3112
Screening	Lookup-30	24/24	0.0163

In deployment, each replay model supplies 12 distinct policy/window predictions for 32 physical cells: 20 pass and 12 fail. Lookup-69 classifies all correctly; Lookup-30 falsely accepts all 12 failures, spanning both negative checkpoints. The two replay models have window-cost MAE of 0.0804 and 0.4657 scenario USD, respectively. Figure 3 exposes PPO-2's startup–cooldown interaction: the 60 s cooldown begins at the capacity-target change. Lookup-30 readiness precedes expiry, leaving time to serve; Lookup-69 and most physical restarts become ready after expiry and can immediately scale down. In the first recovery restart, aligned logs record launch at 120.01 s, readiness at 223.89 s, then draining at 224.01 s without admissions. We manually connect these events to the queued requests and deadline failures; across six runs, 72 of 73 restarts longer than 60 s drain within two seconds of readiness. Long startup alone is insufficient: PPO-1 passes despite 78 of 84 restarts exceeding 60 s (Supplement Section G.2).

The earlier 38-cell mixed-policy study contains rule, guard, and PPO arms on a separate runtime. Lookup-69 also has higher feasibility agreement and lower cost MAE there (Table 6); the campaigns are not pooled. The reversal motivates validating replay models on each deployment cohort rather than selecting one by an earlier agreement score.

Both training environments yield passing and failing deployment checkpoints. The screening study tests Lookup-69 on six previously undeployed checkpoints in a new execution context. From 90 eligible candidates, Lookup-69 predicts 67 feasible on both known windows and rejects 23. A fixed ordering selects three distinct trajectories from each group; Lookup-30 does not enter selection. All six pass every physical run (two windows, two repeats), as do both all2 anchors. Lookup-69 therefore falsely rejects three feasible candidates, while Lookup-30 accepts all six. Across the 24 policy cells, Lookup-30 also has lower cost MAE (Table 6), reversing the deployment-study ordering. In these selected cases, earlier agreement did not transfer to new checkpoints and execution conditions. The supplement records selection

and execution details.

Host-calibrated replay remains insufficient on two continuous BurstGPT days with variable-length budgets (5,241–6,008 online arrivals each, 86,700 s, dual-A100 hosts). All four primary dynamic runs meet online deadlines, but only D2 H+guard meets the joint contract. Share, Lookup-30, and Local select H+guard for both days. On D1, a 45.4 s startup exceeds the guard's frozen 34.1 s p95 safety estimate; one offline job finishes 6.1 s late during drain. The D1 guarded repeat misses eleven deadlines, ten before drain. Both all2 anchors pass. Lookup-69 abstains by falsely rejecting D2 H+guard (Supplement Section P).

These errors also affect deployment choices. Retrospectively requiring predicted P1+ in every window, then minimizing mean predicted cost, selects PPO-2 in the deployment study under Lookup-30; it fails all six physical runs. In the engine study, Lookup-69 selects a feasible policy costing 58.95% more than the lowest-cost feasible policy in the tested pool, which Lookup-30 and Lookup-SG select. The chosen PPO uses 1759.39 versus the guard's 920.79 occupied GPU seconds, with online API routing of 27.83% versus 61.23% (equal-window means); the gap reflects this resource–API trade-off under the scenario ledger. Across a 3×3 GPU/API price grid, the deployment study's failing choice persists and the engine study's gap ranges from 16.56% to 80.17%. Supplement Section L reports choices and ties.

8.3 Prospective HTTP policy selection

We freeze the eight-candidate pool, HTTP service profiles, predictions, and choices before 36 physical executions. All cells are technically valid; 11,137 offloads complete with matched client/server receipts and no HTTP errors. Local selects H+guard, which meets every deadline in all four executions and is cheapest among seven candidates feasible in all four runs (Table 7). The historical proxy reuses the archived A100 startup estimate of 69.219 s without remeasurement on A800. With the same service profile, it predicts three offline misses on steady, excluding H+guard, and selects PPO-1, costing 19.87% more (Supplement Section R). The 30 s predictor and timer diagnostic also select H+guard. In H's two failures, the last offline job remains undispatched with local capacity off: successful online offload cannot replace locally eligible capacity.

8.4 Transfer across serving contexts

The engine study tests whether the evaluation contract can carry unchanged policies to SGLang and still distinguish feasible from infeasible executions. The six arms are H, H+guard, the deployment study's PPO-1/PPO-2, and two screening checkpoints selected by lowest historical mean cost within Lookup-69's accepted and rejected groups. All 26 runs are technically valid. The all2, guarded, and PPO runs satisfy P1+; H meets every deadline in recovery but leaves one offline job unfinished in each steady repeat.

Table 7. Frozen HTTP choices and physical outcomes. Guard R/S is predicted P1+ in recovery/steady. Pass counts cover both windows and repeats; mean system cost includes two fixed provider GPUs, in scenario USD.

Replay	Guard R/S	Choice	Pass	Cost
Local	yes/yes	H+guard	4/4	5.2075
69 s	yes/no	PPO-1	4/4	6.2421

Table 8. Engine study: frozen predictions versus 24 SGLang policy runs. False accepts are out of two infeasible runs; false rejects out of 22 feasible runs. Cost MAE includes the declared deadline penalty, in scenario USD.

Replay	Agreement	False accepts	False rejects	Cost MAE
Lookup-69	16/24	0/2	8/22	0.5420
Lookup-30	24/24	0/2	0/22	0.0468
Lookup-SG	24/24	0/2	0/22	0.0427

The historical Lookup-69 proxy falsely rejects eight feasible runs, whereas Lookup-30 and Lookup-SG match all 24 outcomes, including H's two failures (Table 8). Measuring startup locally slightly lowers cost error relative to Lookup-30 but leaves feasibility judgments unchanged. Each model supplies 12 predictions, each compared with two physical repeats.

The arrival study tests this calibration on additional arrivals without refitting. On three new constructed windows, all 24 policy runs are feasible: Lookup-69, Lookup-30, and Lookup-SG agree on 14/24, 22/24, and 24/24, respectively. Lookup-SG thus avoids the false rejections here.

The 14B study tests remeasured lifecycle timing in Qwen2.5-14B-Instruct/vLLM. H fails all four runs; H+guard passes all four. Replacing the historical startup with the measured value raises agreement from 4/8 to 8/8 and lowers cost MAE from 0.4105 to 0.0668 scenario USD. A retrospective startup-only sweep finds non-monotonic agreement: event replay links longer startup to cooldown pressure, changed targets, and later deadlines. The supplement retains the full grid, including disagreement near the measured value.

The AWQ-Marlin cells transfer the contract to quantized Qwen2.5-7B/vLLM. All four all2 anchors pass, and AWQ-Local matches all eight policy outcomes with zero target disagreements (occupancy/cost MAE: 2.745 GPU seconds/0.00436 scenario USD). AWQ-Hist's 69.219-second startup proxy falsely rejects both feasible H+guard steady repeats (6/8 correct); the 30-second AWQ-30 matches 8/8 with roughly $3\times$ the occupancy error. Both H/steady fail-

Table 9. Formal held-out results. Each arm has three repeats per window; H has 15 offline deadline misses across its three recovery repeats.

Arm	tech.	offline	online
all2	9/9	9/9 (120/120)	9/9
all1	9/9	9/9 (120/120)	9/9
H	9/9	6/9	9/9
H+guard	9/9	9/9 (120/120)	9/9

ures repeat the same unfinished offline request; details are in the supplement.

Capacity pressure extends validation to four active replicas. Two studies on one four-A800 host compare two/four TP1 replicas with doubled arrivals at four GPUs (36 cells). On the original windows, Lookup-69 ties H and H+guard, although H fails three of four four-replica runs and the guard passes all four. With constructed offline pressure, all four dynamic runs at four replicas pass; GPUs 2 and 3 serve 302–332 and 54–68 requests after restarting. Lookup-69 rejects both candidates and abstains. Local matches all four feasibility labels; the physical cost ranking reverses between repeats. H+guard meets every deadline at 54.94–58.60% lower cost than matched all4 references (Supplement Section Q).

8.5 Control case study: feasibility and cost

The guard provides a rule-based control case alongside the learned policies. Table 9 summarizes the original held-out study. All2 and all1 satisfy the registered completion target. H is technically valid but fails the P1 gate in all three recovery repeats (116/120, 113/120, and 116/120 offline on time). In that held-out study, the guard repairs this failure without changing the workload or the locked model.

H+guard reduces window cost relative to all1 by 21.96%, 48.19%, and 58.08% across steady, recovery, and burst-offline (equal-window mean 42.74%; including setup/cleanup, 38.86%). This exceeds the registered 25% mean P2 target, although steady does not. Local's locked recovery prediction flags four unfinished offline jobs; physical H repeats leave four, seven, and four.

8.6 Latency–cost trade-off

Resource savings consume online latency margin: per-repeat p99 is about 1.6–2.6 s for always-on references and 15–19 s for both dynamic arms (supplementary tail table). Requests routed to the synthetic API sink wait at least 10 s before admission; the guard protects offline deadlines without removing this delay. All observed p99 values remain below the registered 60 s online deadline, with less latency margin than the always-on references.

Table 10. Guard activity by recovery repeat. Ticks are window seconds; slack is the minimum pending-offline time to deadline (seconds), before subtracting service time.

Repeat	risk ticks	target raises	tick range	slack range
1	30	1	1982–2011	29–58
2	0	0	–	–
3	29	1	2027–2055	30–58

8.7 Guard activation and mechanism

The guard checks deadline risk using the visible queue, running-job ages, lifecycle delay, and concurrency-four service estimate. Equation 4 retains risk IDs until dispatch or removal from the pending queue and distinguishes risk from a target raise. In the held-out guard study, the logs contain 59 risk-active ticks, all in recovery: the guard raised the target above H twice; remaining risk ticks only maintained that target. Its steady and burst-offline runs have no risk-active ticks. In the 14B study, active guard runs in recovery and steady restart capacity for tail queues below H's scale-out threshold (supplement). Table 10 reports recovery counts and slack ranges; Figure 4 shows the repeat-1 target and queue trajectories.

8.8 Price sensitivity and external checks

Repricing fixed event streams over 27 window–GPU–API price cells leaves recovery and burst savings positive, but steady can become negative (-39.59%); its break-even API multiplier is 1.004. The supplement retains the full table and scenario assumptions.

External checks cover queue-target (B-HPA), Holt-predictive (B-PRED), and queueing-based (B-MMC) baselines, Llama, and Azure arrivals (supplement). Each baseline meets all deadlines in nine runs; H+guard costs less than B-HPA on two known Qwen windows, with higher API use and queue exposure. Retrospective replay of 24 Azure runs gives four archived lookup-based models identical 21/24 agreement, but Lookup-69 falsely accepts three runs while the others reject three feasible runs. H's physical repeats have mixed outcomes in both windows. Identical agreement scores thus conceal opposite deployment risks.

9 Discussion

A predictor can accept failing PPO deployments yet reject feasible alternatives in another cohort. The prospective HTTP study validates a choice frozen before deployment, with every deadline met; its historical-proxy alternative costs 19.87% more. Physical validation should check admission risk and the cost of excluding feasible alternatives.

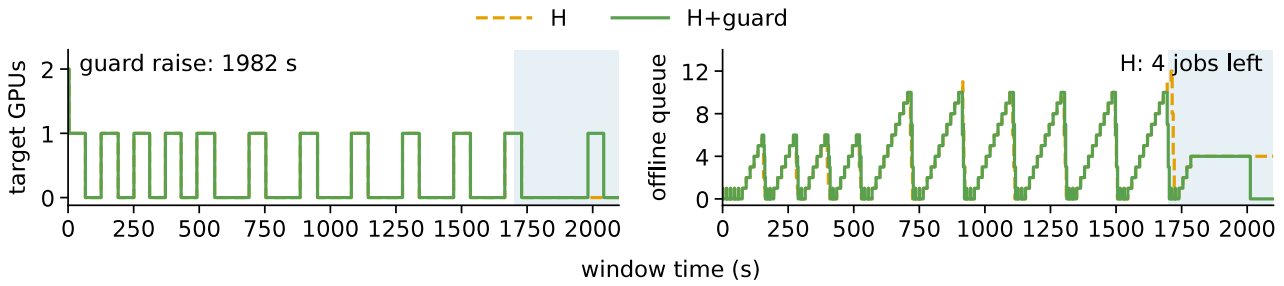


Figure 4. Recovery target and offline queue in repeat 1. The shaded final interval contains the guard raise at 1982 s, where the target trajectories first diverge. H leaves four offline jobs queued, while the guard clears them. Across all three H repeats, 15 jobs miss their deadlines.

9.1 Validating predictions across levels

Validation separates setup from scale-out and aligns launch, readiness, draining, release, capacity targets, and cooldown expiry. In the deployment study, readiness after expiry permits immediate scale-down where replay predicts protection. Action trajectories reveal temporal misalignment that aggregate resource error can conceal.

The guard uses observed queues and frozen safety estimates to repair H's held-out recovery failure. H never reads simulator startup parameters; correcting replay endpoints and guarding physical execution have distinct roles.

9.2 Resource and latency trade-offs

Elastic policies exchange online latency margin for lower occupied capacity. Compare savings under a common feasibility and API/queue/lifecycle ledger. B-HPA, price, and Llama/Azure checks establish the tested economic and workload boundaries; meeting one SLO does not establish tighter-SLO suitability.

Practical validation. Our screening reversals motivate validating both accepted and rejected candidates after rule development or PPO training. Freeze the pool and predictions, then compare physical feasibility and cost under one ledger. Retain selection ties and report each repeat when physical rankings reverse. Request-aligned service, action, queue, and resource traces explain failures that aggregate prediction error alone misses.

10 Scope and Limitations

Earlier cohorts use a five-second synthetic API timer. The prospective study generates actual responses on same-host, same-weight HTTP providers and charges their fixed capacity; it does not establish commercial API latency, cross-model quality equivalence, or provider billing savings. GPU release measures occupied resources, not power or energy.

Evidence spans Qwen 7B/14B and Llama 8B, two engines, full-day inputs, quantization, and up to four same-host replicas. Repeated fixed inputs and selected days/checkpoints limit population inference. Physical repeats reuse frozen predictions; all-feasible cohorts do not test failure detection. The guard's heuristic finish estimates are not proven completion-time upper bounds and provide no global deadline guarantee. The studies do not establish heterogeneous scaling or universal rankings.

11 Conclusion

FidelityLoop links replay-based policy selection to physical deadline and resource outcomes. Startup–cooldown interactions and errors in capacity timing can admit failing policies or exclude cheaper feasible alternatives despite accurate service or aggregate-resource estimates. We apply the same contract to rule-based and learned policies across two serving engines and with an external service model. Prospective HTTP execution confirms the feasibility and cost consequence of a choice frozen before deployment. Traces and CPU tools support reuse. Elastic-serving replay needs validation of lifecycle timing, online/offline feasibility, and action trajectories before deployment.

REFERENCES

Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. Vidur: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 351–366, 2024a. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B., Tumanov, A., and Ramjee, R. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 117–134, Santa Clara, CA, July 2024b. USENIX Association. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

Crankshaw, D., Sela, G.-E., Mo, X., Zumar, C., Stoica, I., Gonzalez, J., and Tumanov, A. InferLine: Latency-Aware Provisioning and Scaling for Prediction Serving Pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pp. 477–491, 2020. doi: 10.1145/3419111.3421285. URL https://doi.org/10.1145/3419111.3421285.

Fu, Y., Xue, L., Huang, Y., Brabete, A.-O., Ustiugov, D., Patel, Y., and Mai, L. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 135–153, 2024. URL https://www.usenix.org/conference/osdi24/presentation/fu.

Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., and Mace, J. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 443–462, 2020. URL https://www.usenix.org/conference/osdi20/presentation/gujarati.

Kabakibo, H. S., Trivedi, A., and Wang, L. Breaking the Ice: Analyzing Cold Start Latency in vLLM. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1187–1205, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/29416b66c2149872b9d1415a3fd2c5e0-Abstract-Conference.html.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 118–132, 2024. doi: 10.1109/isca59077.2024.00019. URL https://doi.org/10.1109/isca59077.2024.00019.

Qiao, Y., Anzai, S., Yu, S., Ma, H., Yang, S., Wang, Y., Kim, M., Wu, Y., Zhou, Y., Xing, J., Gonzalez, J. E., Stoica, I., and Xu, H. ConServe: Fine-grained GPU harvesting for LLM online and offline co-serving. *arXiv preprint arXiv:2410.01228*, 2024. URL https://arxiv.org/abs/2410.01228.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 173–191, 2024. URL https://www.usenix.org/conference/osdi24/presentation/sun-biao.

Wang, Y., Chen, Y., Li, Z., Kang, X., Tang, Z., He, X., Guo, R., Wang, X., Wang, Q., Zhou, A. C., and Chu, X. BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems, 2024. URL https://arxiv.org/abs/2401.17644v3.

Yang, M., Zhang, Z., Wu, M., Yan, J., Sun, H., and Chang, L.-W. Charon: A Unified and Fine-Grained Simulator for Large-Scale LLM Training and Inference. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 94–107, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/dbc8ce0fdfcd55172d73fb05dbae07fc-Abstract-Conference.html.

Zhang, C., Yu, M., Wang, W., and Yan, F. MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving. In *2019 USENIX Annual Technical Conference*, pp. 1049–1062, 2019. URL https://www.usenix.org/conference/atc19/presentation/zhang-chengliang.

Zhang, G., Guo, W., Tan, Z., Guan, Q., and Jiang, H. KISS: A GPU-Aware Kubernetes Inference Simulator with RL-Based Auto-Scaling, 2025a. URL https://arxiv.org/abs/2507.07932.

Zhang, G., Vippagunta, S., Nandagopal, R., Raman, S., Xu, J., Pfeiffer, M., Chatterjee, S., Tan, Z., Guo, W., and Jiang, H. AAPA: An Archetype-Aware Predictive Autoscaler with Uncertainty Quantification for Serverless Workloads on Kubernetes, 2025b. URL https://arxiv.org/abs/2507.05653.

Zhang, H., Tang, Y., Khandelwal, A., and Stoica, I. SHEPHERD: Serving DNNs in the Wild. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*, pp. 787–808, 2023. URL https://www.usenix.org/conference/nsdi23/presentation/zhang-hong.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient Execution of Structured Language Model Programs. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 62557–62583. Curran Associates, Inc., 2024. doi: 10.52202/079017-2000. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf.

AI USAGE DISCLOSURE

We used OpenAI Codex and Anthropic Claude extensively to assist with research planning, literature search, experimental design, code implementation and debugging, synthetic workload construction, data processing, analysis, visualization, documentation, and manuscript drafting and revision. These tools also helped develop execution and validation scripts, inspect experimental outputs, check numerical and narrative consistency, and propose supplementary experiments. We used AI-assisted code to construct controlled workloads and process recorded traces. The paper distinguishes synthetic inputs from measured execution outcomes; the reported physical measurements were obtained from experimental runs. We directed the research, made the final decisions about experimental protocols and claims, and reviewed and verified the implementations, experimental evidence, analyses, citations, and manuscript. These tools are not authors. We retain full responsibility for the correctness, originality, attribution, and integrity of this submission.